

Rapport technique de récupération / reconstruction

Wallix Agent — analyse forensique de l'APK fonctionnel

Élément	Valeur
APK	f076cdd0-2ead-429f-a8dd-73d4c9d11c1a.apk
Taille	70.88 MiB
SHA-256	d6c509d00cba79c82e5ee63f33ce8ed2143d301a986141e35525b1017f769ec0
Moteur	Flutter / Dart AOT
ABI	arm64-v8a, armeabi-v7a, x86_64
Date	21 août 2026
Objet	Cartographie architecture, API, données, assets et stratégie de reconstruction

Verdict exécutif : l'APK est une excellente base de récupération. Il conserve 58 chemins Dart propres à l'application, des noms de classes/méthodes, 10 endpoints API, des clés métier, des assets, des polices, les plugins et de nombreuses chaînes fonctionnelles. Le code Dart original n'est cependant pas stocké sous forme de fichiers source dans une release AOT : la suite du travail est donc une reconstruction guidée par preuves.

1. Méthodologie et niveau de preuve

L'APK a été inspecté comme archive ZIP puis le binaire Flutter **lib/arm64-v8a/libapp.so** a été analysé par extraction de chaînes. Les variantes armeabi-v7a et x86_64 contiennent également libapp.so. Les manifestes Flutter, assets, polices et bibliothèques natives ont été inventoriés.

Observé = présent littéralement dans l'APK. **Fortement inféré** = plusieurs indices convergents. **À confirmer** = nécessite désassemblage ciblé ou instrumentation réseau.

Important — api_config : votre précision est intégrée : **api/api_config.dart n'a pas été retrouvé** parmi les 58 chemins **package:wallix/**. Il doit donc être traité comme composant à reconstituer, pas comme fichier observé.

2. Architecture récupérée

```
lib/
  ■■■ main.dart
  ■■■ api/
    ■ ■■■ api_config.dart          # NON OBSERVÉ / À RECONSTITUER
    ■ ■■■ api_default.dart
    ■ ■■■ api_service.dart
    ■■■ data/shared_prefs_helper.dart
    ■■■ firebase_initializer.dart
    ■■■ firebase_service.dart
    ■■■ l10n/                      # FR + EN
    ■■■ model/                    # auth, home, transaction, retrait, etc.
    ■■■ providers/                # homeProvider, notification_provider
    ■■■ screens/                  # 18 écrans
    ■■■ services/                 # 6 services
    ■■■ widgets/                  # 7 widgets
```

La structure suggère une architecture : **API** → **services** → **modèles** → **providers** → **screens/widgets**, avec Provider pour l'état, SharedPreferences pour le stockage local et Firebase/notifications pour le canal push.

3. Inventaire des écrans

Fichier	Responsabilité déduite
splash_screen.dart	Splash / bootstrap
onboarding_screen.dart	Onboarding
login_screen.dart	Authentification agent
access_code_screen.dart	Saisie code sécurité
forgot_code_screen.dart	Récupération code
change_pin_screen.dart	Modification PIN
home_screen.dart	Accueil agent
main_navigation_screen.dart	Navigation principale
history_screen.dart	Historique transactions
transaction_detail_screen.dart	Détail transaction
retraits_screen.dart	Liste / annulation retraits
commissions_screen.dart	Commissions
qr_scanner_screen.dart	Scan QR client
client_confirmation_screen.dart	Confirmation client
profile_screen.dart	Profil agent
notifications_screen.dart	Centre notifications
support_screen.dart	Support
faq_screen.dart	FAQ

4. Endpoints API observés

Base URL littérale observée : **https://cygne.mdkrlabs.dev/api/web/v1**. Les dix routes suivantes sont certaines. Le verbe HTTP exact de chaque route n'est pas certifiable uniquement par les chaînes extraites ; le binaire contient néanmoins **POST** et des indices de requêtes HTTP. Il faut confirmer l'association route→verbe par analyse AOT ciblée ou proxy contrôlé.

Route	Fonction	Certitude
agents/start	Bootstrap/accueil agent	URL observée
agents/operations	Historique / opérations	URL observée
agents/depot	Dépôt	URL observée
agents/init_retrait	Initialisation retrait	URL observée
agents/cancel_retrait	Annulation retrait	URL observée
agents/all_commission	Liste commissions	URL observée
agents/all_retrait	Liste retraits	URL observée
contacts/check_qr	Validation QR client	URL observée
users/connecte_useraccount	Connexion compte utilisateur	URL observée
users/update_password	Mise à jour mot de passe / code	URL observée

5. Flux métier et services

Domaine	Service / modèles	Indices observés
Accueil	home_service.dart; InfosAccueil/InfosResponse	_loadHomeData, total_depots, agentName
Opérations	transaction_service.dart; TransactionResult	transactionHistory, TransactionService
Dépôt	transaction_service.dart; ClientInfo	DEPOT REQUEST/RESPONSE, key_depot, depot_datetime
Retrait	transaction_service.dart; RetraitItem	initRetrait, retrait_datetime
Annulation	transaction_service.dart	CANCEL RETRAIT REQUEST/RESPONSE, key_retraitP
Commissions	commission_response.dart	COMMISSION REQUEST/RESPONSE, getCommissions
Retraits	retrait_item.dart	listRetraits, RetraitListResponse
QR	qr_service.dart; QrCheckResponse	checkQrCode, QR CHECK INFO, nomComplet=
Compte	auth/profile services	connecte_useraccount
Mot de passe/PIN	auth_service.dart	update_password, current_pin, new_pin

6. Clés et paramètres métier

Session / sécurité:
 access_token, client_token, token, Bearer, fcm_token,
 user_pin, current_pin, new_pin, pinCode

Agent:
 agent_name, numero_agent, agent_photo, agentName

Client / QR:
 client_nom, client_telephone, client_token, client_verified,
 qrCode, qr_code, nomComplet

Transactions:
 amount, montant, montant_total, transaction_datetime,
 heure_transaction, date_transaction, type_operation_key,
 transaction_details, depot_datetime, key_depot, total_depots

Retraits / notifications:
 retrait_datetime, key_retraitP, commission, commissions,
 total, unread_notifications_count

Ces chaînes sont des points d'entrée pour reconstruire les DTO JSON. Elles ne prouvent pas toutes que chaque clé est envoyée au serveur : certaines peuvent être UI ou stockage local.

7. Authentification et stockage

Le binaire contient explicitement **Bearer**, **access_token**, **client_token** et **token**. Cela rend fortement probable un mécanisme de header Authorization Bearer. **shared_prefs_helper.dart** et le package **shared_preferences** confirment un stockage local.

Firebase Messaging est actif : **Messaging#getToken**, **onTokenRefresh**, **FCM Token:**, **fcm_token** et **_saveToken** sont observés. Le plugin **flutter_local_notifications** est également embarqué.

8. Modèles de données

Modèle	Preuves / rôle
User / LoginResponse / SimpleResponse	auth, login, session agent
ClientInfo	client_nom, client_telephone, client_token, fromJson
InfosAccueil / InfosResponse	home data, totaux, agent
TransactionItem / TransactionResult	fromJson, fromJson, historique
CommissionResponse	fromJson, getCommissions
RetraitItem / RetraitListResponse	fromJson, listRetraits
QrCheckResponse	fromJson, checkQrCode
NotificationModel / AppNotification	fromJson/fromMap, unread count
ProfileData / SettingsItem	profil et paramètres
OnboardingData	données onboarding

9. Internationalisation et UI

Les fichiers **app_localizations.dart**, **app_localizations_en.dart** et **app_localizations_fr.dart** sont présents. **locale_provider.dart** et **language_bottom_sheet.dart** montrent un changement de langue structuré. Des chaînes françaises et anglaises couvrent les écrans métier.

10. Plugins / dépendances identifiables

```
http
provider
pinpoint
mobile_scanner
country_picker
firebase_core
firebase_messaging
flutter_local_notifications
shared_preferences
url_launcher
iconsax_flutter
intl
nested
path
collection
flutter_native_splash
```

Le scanner utilise **mobile_scanner**. **url_launcher** est utilisé pour des actions externes ; l'APK contient notamment <https://wa.me/> et supportagentwallix@gmail.com.

11. Assets et polices

Asset	Dimensions
assets/images/onboarding_1.png	482×518
assets/images/onboarding_2.png	456×547
assets/images/logo_wallix.jpeg	1024×1024
assets/images/logo_native.png	1024×1024
assets/images/splash.png	720×1447
assets/images/depot.jpeg	48×48
assets/images/retrait.jpeg	32×32

FontManifest.json confirme TestSöhne en poids 300/400/600/700, plus MaterialIcons, FlutterIcons et CupertinoIcons.

```
TestSöhne 300 -> TestSöhne-Leicht.otf
TestSöhne 400 -> TestSöhne-Buch.otf
TestSöhne 600 -> TestSöhne-Kräftig.otf
TestSöhne 700 -> TestSöhne-Fett.otf
```

12. Android / build

La classe native **com/agent/wallix/MainActivity** est présente dans le bytecode Android. Trois ABI sont embarquées : arm64-v8a, armeabi-v7a, x86_64. Le binaire AOT contient une signature de build release avec **no-code_comments** et **no-dwarf_stack_traces_mode**.

Le chemin de build historique retrouvé est :

file:///D:/BERNARD/salle_e/wallix/.dart_tool/flutter_build/dart_plugin_registrant.dart. La metadata APK indique **NO_SUPPORTED_VCS_FOUND**, donc aucun historique Git exploitable n'est présent dans cette archive.

13. api_config.dart — reconstruction

Votre ajout est important : **api_config.dart** doit être ajouté au nouveau projet. Il n'a pas été observé dans les 58 chemins Dart du snapshot. La seule constante certaine est la base URL ci-dessous.

```
// valeur OBSERVÉE
https://cygne.mdkrlabs.dev/api/web/v1

// architecture RECOMMANDÉE, à confirmer
lib/api/api_config.dart
- baseUrl
- timeouts éventuels
- headers communs éventuels
- constantes d'API éventuelles

lib/api/api_default.dart
- comportement HTTP par défaut

lib/api/api_service.dart
- requêtes
- auth headers
- JSON encode/decode
- gestion erreurs
```

Ne pas inventer les autres constantes avant l'analyse ciblée de api_service.dart et des services : elles doivent être marquées *observées* ou *inférées*.

14. Récupérable / perdu

Élément	État
Arborescence Dart	Très bien récupérable — 58 chemins
Classes / méthodes	Très bien récupérables — symboles/chaînes AOT
Endpoints	Très bien récupérables — 10 URLs littérales
Assets / fonts	Directement récupérables
Plugins	Très bien récupérables
Textes / FR / EN	Très bien récupérables
Clés JSON	Partiellement à très bien
Verbes HTTP exacts par route	À confirmer
Corps JSON exacts	À reconstruire
Code Dart source original	Non disponible tel quel
Commentaires / formatage	Perdus
api_config.dart	Non observé — à reconstituer

15. Plan de reconstruction recommandé

- P0** : figer l'APK original et conserver le SHA-256.
- P0** : extraire définitivement assets, fonts et manifestes Flutter.

- 3 **P0** : créer le squelette lib/api, data, l10n, model, providers, screens, services, widgets.
- 4 **P0** : reconstituer api_config.dart avec la base URL observée.
- 5 **P0** : analyser auth_service.dart et transaction_service.dart en priorité.
- 6 **P1** : reconstruire DTO/JSON : User, LoginResponse, ClientInfo, Transaction*, Retrait*, CommissionResponse, QrCheckResponse.
- 7 **P1** : confirmer verbes HTTP, headers et payloads par analyse AOT ou instrumentation réseau.
- 8 **P1** : reconstruire home_service, qr_service et profile_service.
- 9 **P2** : refaire les écrans et navigation puis comparer comportement/visuel à l'APK.
- 10 **P2** : réintégrer Firebase Messaging et notifications locales.

16. Inventaire exhaustif des 58 fichiers

#	Chemin
1	api/api_default.dart
2	api/api_service.dart
3	data/shared_prefs_helper.dart
4	firebase_initializer.dart
5	firebase_service.dart
6	l10n/app_localizations.dart
7	l10n/app_localizations_en.dart
8	l10n/app_localizations_fr.dart
9	main.dart
10	model/auth/login_response.dart
11	model/auth/simple_response.dart
12	model/auth/user.dart
13	model/client_info.dart
14	model/commission_response.dart
15	model/home/infos_accueil.dart
16	model/home/infos_response.dart
17	model/notification_model.dart
18	model/onboarding_data.dart
19	model/operation_history_response.dart
20	model/profile_data.dart
21	model/qr_check_response.dart
22	model/retrait_item.dart
23	model/settings_item.dart
24	model/transaction_item.dart
25	model/transaction_result.dart
26	providers/homeProvider.dart
27	providers/notification_provider.dart
28	screens/access_code_screen.dart
29	screens/change_pin_screen.dart
30	screens/client_confirmation_screen.dart

#	Chemin
31	screens/commissions_screen.dart
32	screens/faq_screen.dart
33	screens/forgot_code_screen.dart
34	screens/history_screen.dart
35	screens/home_screen.dart
36	screens/login_screen.dart
37	screens/main_navigation_screen.dart
38	screens/notifications_screen.dart
39	screens/onboarding_screen.dart
40	screens/profile_screen.dart
41	screens/qr_scanner_screen.dart
42	screens/retraits_screen.dart
43	screens/splash_screen.dart
44	screens/support_screen.dart
45	screens/transaction_detail_screen.dart
46	services/auth_service.dart
47	services/home_service.dart
48	services/locale_provider.dart
49	services/profile_service.dart
50	services/qr_service.dart
51	services/transaction_service.dart
52	widgets/dotted_loader.dart
53	widgets/language_bottom_sheet.dart
54	widgets/logout_confirm_dialog.dart
55	widgets/profile_header_card.dart
56	widgets/settings_section_widget.dart
57	widgets/settings_tile.dart
58	widgets/t_text.dart

17. Annexe — preuves techniques

Endpoints littéraux observés :

```
https://cygne.mdkrlabs.dev/api/web/v1/agents/start
https://cygne.mdkrlabs.dev/api/web/v1/agents/operations
https://cygne.mdkrlabs.dev/api/web/v1/agents/depot
https://cygne.mdkrlabs.dev/api/web/v1/agents/init_retrait
https://cygne.mdkrlabs.dev/api/web/v1/agents/cancel_retrait
https://cygne.mdkrlabs.dev/api/web/v1/agents/all_commission
https://cygne.mdkrlabs.dev/api/web/v1/agents/all_retrait
https://cygne.mdkrlabs.dev/api/web/v1/contacts/check_qr
https://cygne.mdkrlabs.dev/api/web/v1/users/connecte_useraccount
https://cygne.mdkrlabs.dev/api/web/v1/users/update_password
```

Chaînes sécurité/session :

```
access_token
client_token
token
Bearer
fcm_token
user_pin
current_pin
new_pin
verifyPin
getPinCode
```

Chaînes métier :

```
DEPOT REQUEST:
DEPOT RESPONSE:
CANCEL RETRAIT REQUEST:
CANCEL RETRAIT RESPONSE:
COMMISSION REQUEST:
COMMISSION RESPONSE:
QR CHECK INFO: nomComple=
TransactionItem.fromApi
TransactionResult.fromJson
CommissionResponse.fromJson
RetraitItem.fromJson
RetraitListResponse.fromJson
QrCheckResponse.fromJson
ClientInfo.fromJson
```

18. Limites et prochaines analyses

Une chaîne dans un snapshot AOT peut représenter une constante UI, une clé JSON, une clé SharedPreferences, un nom interne ou un symbole. Il faut donc distinguer preuve et hypothèse. Le code source original, les commentaires et la mise en forme ne sont pas récupérables tels quels depuis une release AOT.

La prochaine passe la plus rentable est une analyse ciblée de libapp.so autour des méthodes propres à **ApiService**, **ApiDefault**, **AuthService**, **TransactionService**, **HomeService**, **QrService** et **ProfileService**. Elle doit viser : verbes HTTP, headers, payloads JSON, clés SharedPreferences et règles de parsing.

Conclusion : l'APK fournit une base techniquement solide pour reconstruire l'application. La perte du dépôt source n'est pas équivalente à une perte totale du projet : architecture, contrat API, ressources et une part substantielle des métadonnées applicatives sont encore exploitables.